

태양전지 탑재 차량의 자동 최적 주차

융합캡스톤디자인2 최종 발표

5조

- 201720702 유효조
- 201820834 마한성
- 201820911 박현재
- 202021025 안준영

목차

#01

프로젝트 소개

- 구현 기능 및 목표
- 역할 분담

#02

의의 및

- 전기차의 확대
- 태양 전지의 발전 가능성

#03

설계

- 전원부 설계
- 구동부 설계

#04

결과

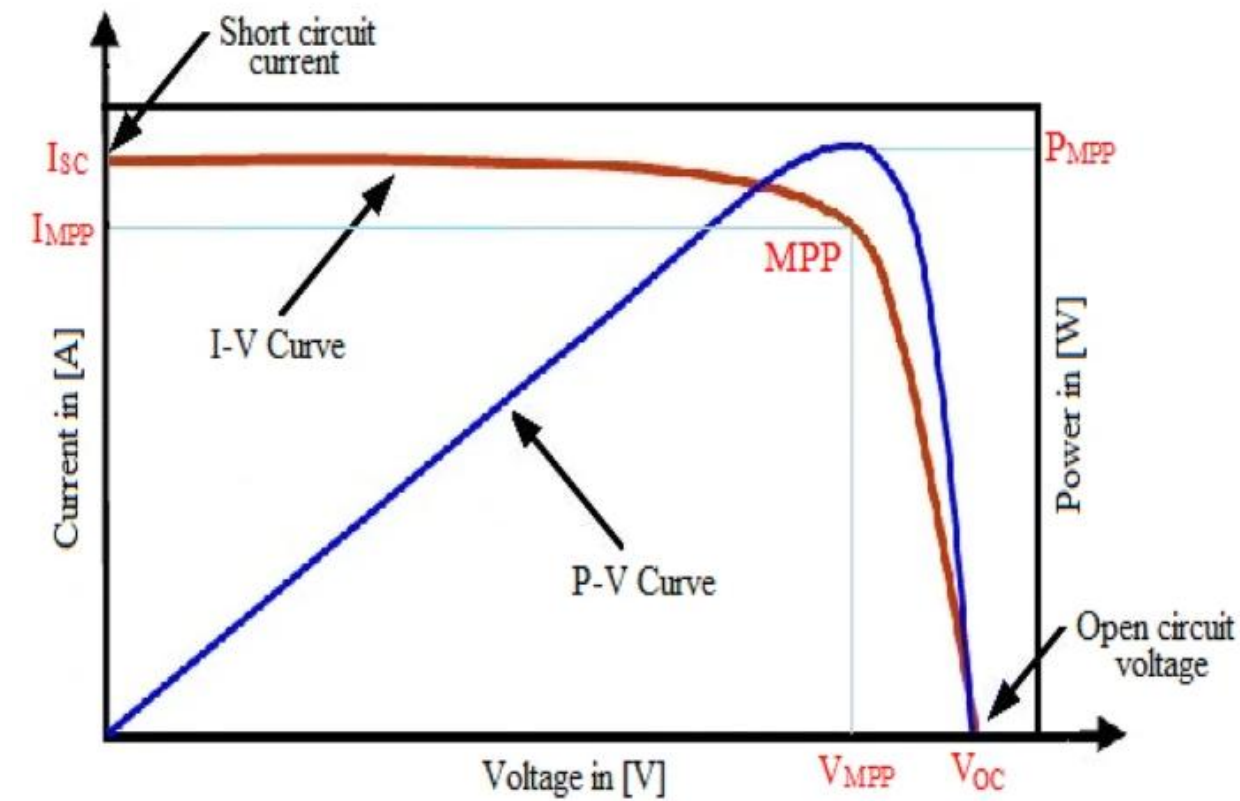
#05

결론 및 Q&A



태양 전지의 전력원 활용

- 전력원으로 활용
- 배터리 전기차에 적합한 환경
- 주행거리 연장 효과
- 친환경 에너지 활용



MPPT 컨트롤러 도입

- P&O 방식 알고리즘 적용
- 실시간 최대 전력점 추적
- 다양한 일조량에 대응
- 전력 변환 효율 향상



최적 주차 위치 이동

- 차량은 대부분 주차 상태
- 태양광 최대 지점 선정
- 발전량 증가 효과

안준영 (팀장)

- 차량 구동 제어부 및 센싱 설계 담당
- 모터 PID 제어기 튜닝
- Pure Pursuit 알고리즘 구현
- 전체 프로젝트 관리 및 조율

유효조

- Pure Pursuit 담당
- 차량 구동 제어부 및 센싱 설계
- 모터 PID 제어기 튜닝
- 오도메트리 구현

마한성

- 태양전지 제어 MPPT 담당
- 전원부 제어기 설계
- 무선 통신 설계
- P&O 알고리즘 구현 및 최적화

박현재

- 전원부 제어기 설계 담당
- 태양전지 제어 MPPT 보조
- 무선 통신 설계
- 전력 변환 회로 구현

페로브스카이트 및 박막형 태양 전지

- 페로브스카이트 태양 전지: 저비용 고효율 차세대 태양전지 기술
- 박막형 태양 전지: 유연성과 경량화로 다양한 표면에 적용 가능
- 효율 향상 및 생산 비용 절감으로 상용화 가능성 증대

1↓

자동차에서의 태양 전지 활용

- 현대차의 솔라루프 시스템: 보조 전력원으로 활용
- 미국 스타트업 앵테라의 3륜 태양광 전기차 개발
- 차량 표면 활용 태양전지로 주행거리 연장 가능

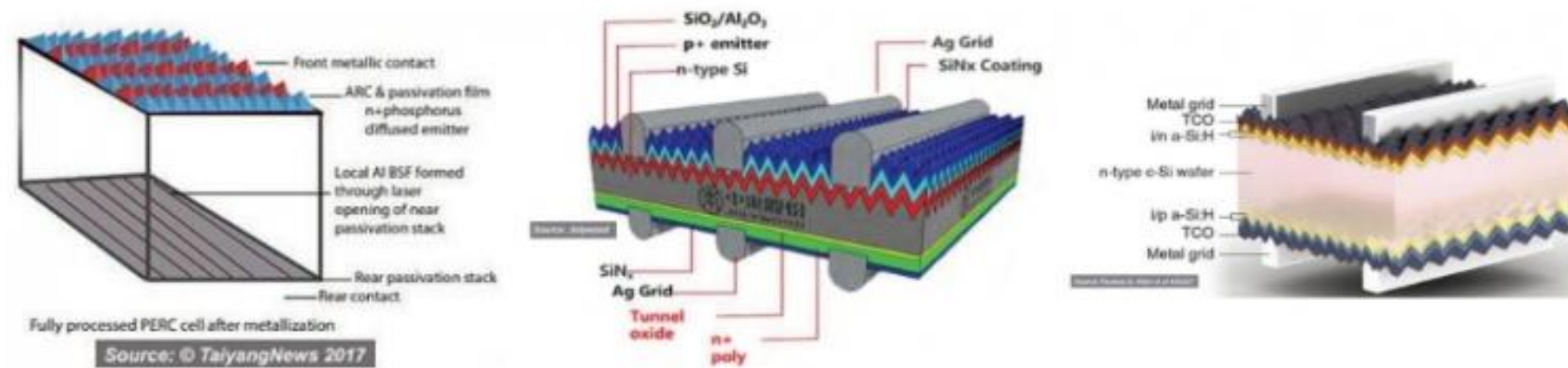
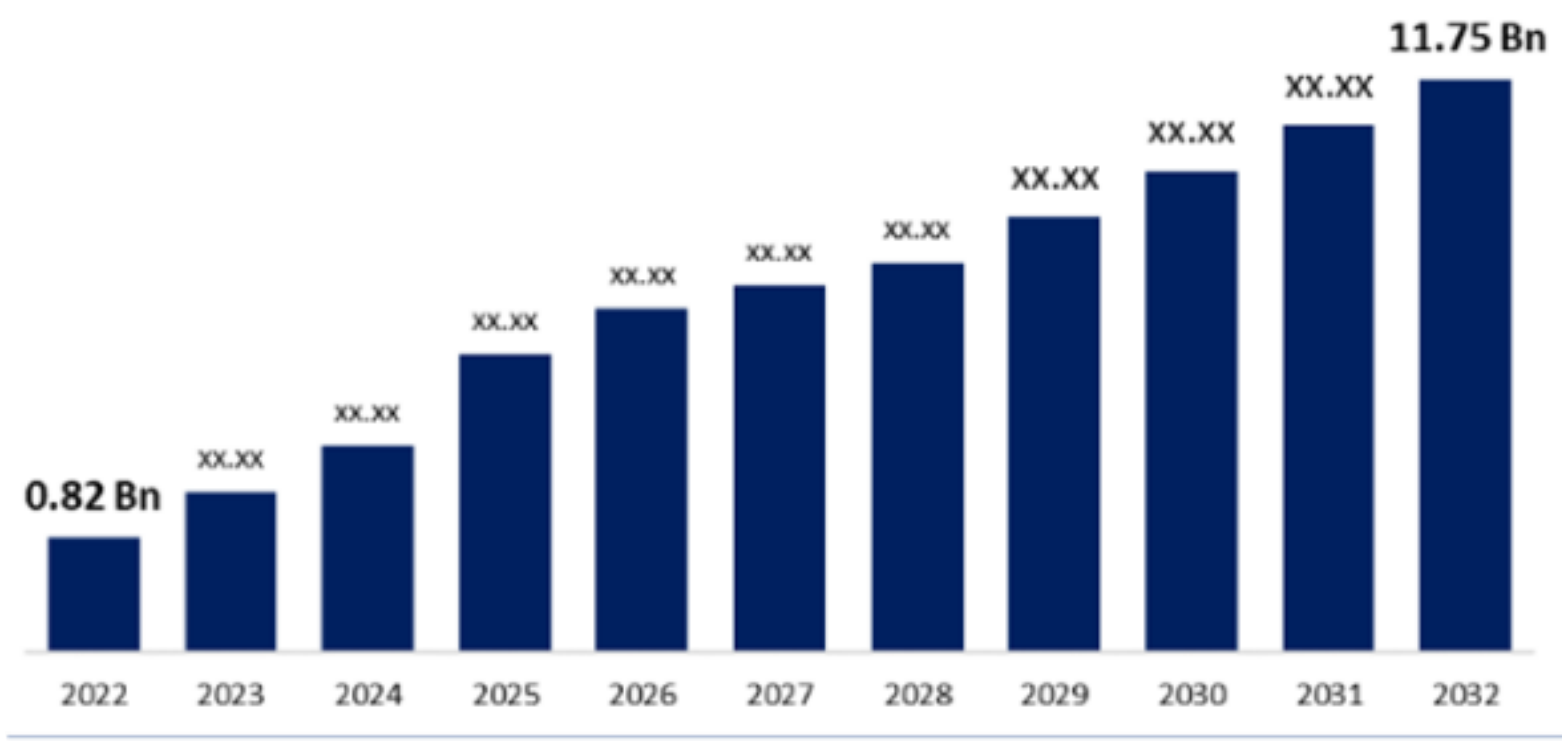


그림 1. 고효율 결정질 실리콘 태양전지 구조 [출처=Taiaing News]



Global Perovskite Solar Cell Market



시스템구성및구동 원리



전기차 시장의 성장

- 2022년 전기 자동차 총 802만대 판매, 전체 완성차 판매량의 약 10% 차지
- 테슬라 모델Y, 상해GM 우링 홍광 MINI, 테슬라 모델3 등 다양한 모델 출시
- 현대차그룹도 아이오닉5, EV6 등을 개발 및 양산하며 꾸준히 투자 중
- 전기차 시장은 지속적으로 성장하는 추세

태양전지와 전기차의 결합

- 현대차의 솔라루프 기술 적용으로 보조 전력 공급
- 미국 스타트업 애플테라의 3륜 태양광 전기차 개발
- 태양전지를 통한 주행거리 연장 효과
- 주차 중 배터리 충전으로 에너지 효율 향상
- 페로브스카이트, 박막형 태양 전지 등 신기술 적용 가능성

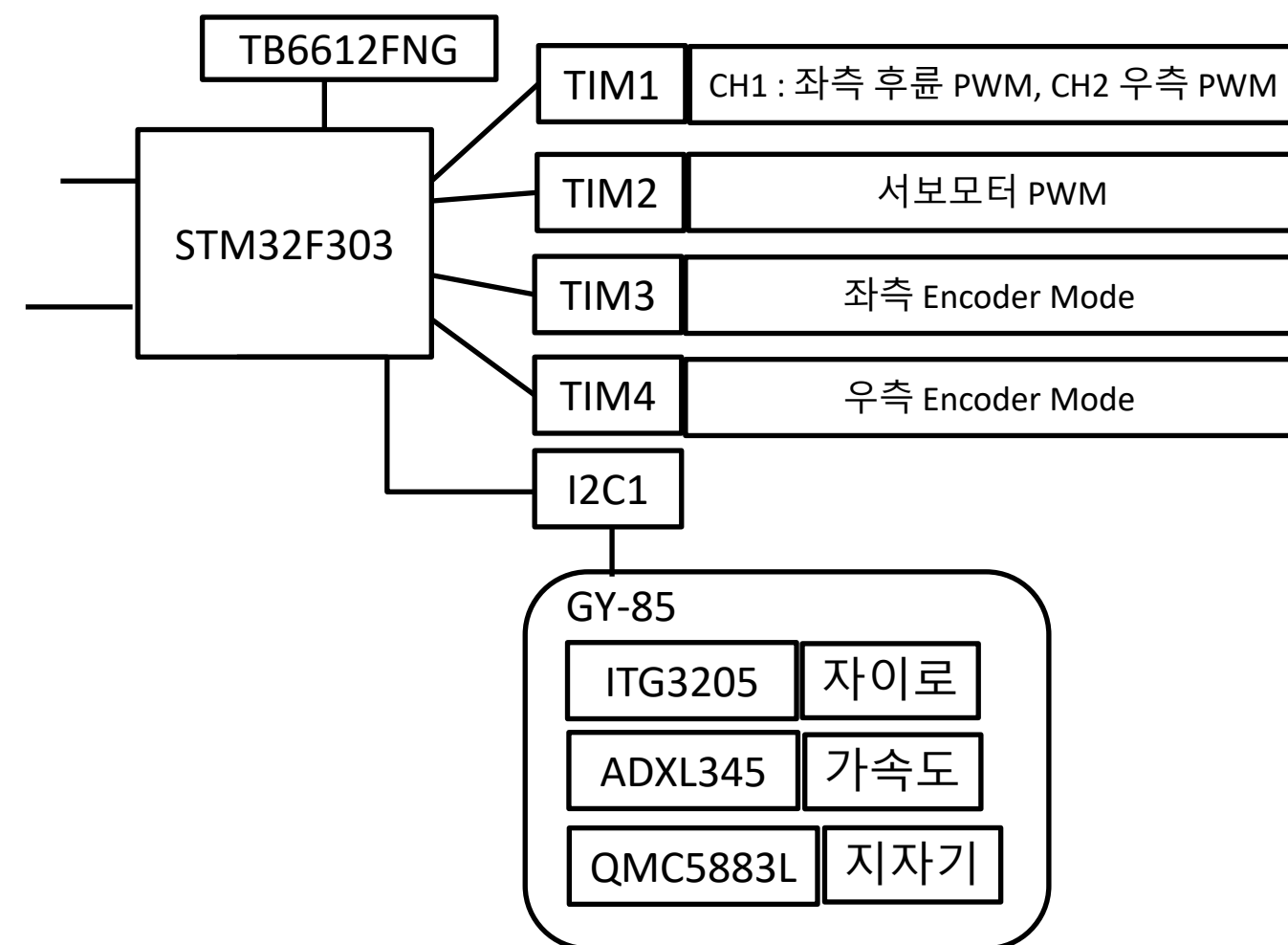
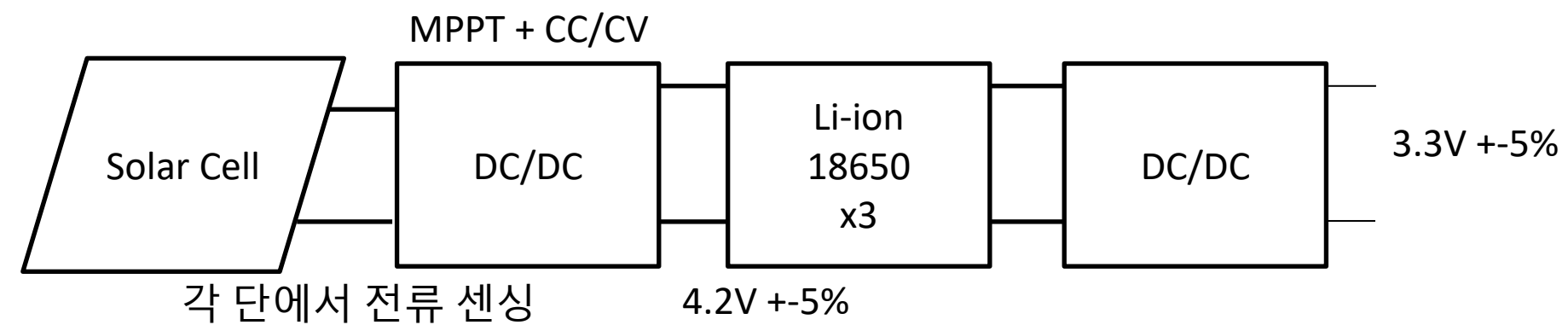
전원팀 목표

- DC/DC 컨버터 회로 설계
- P&O MPPT 제어를 통한 전력 제어
- 출력 전압 리플 5% 미만 달성
- 도입 전/후 20% 이상 전력 효율 향상

구동팀 목표

- 차량의 주차장 내 최대 태양광 노드로 자동 이동
- 3cm 이내 도달 오차 달성
- 부드러운 주행 구현

전력 변환 시스템



- 태양전지 → AC Flyback → 리튬이온 18650 배터리 충전 시스템
- 액티브 클램프 플라이백 회로 설계 및 제어기 설계
- 배터리 → Buck → MCU 전원 공급 시스템
- Buck 회로 설계 및 제어기 설계
- MPPT 알고리즘 구현으로 최대 전력점 추적
- 각 단에서 전류 센싱으로 효율 모니터링
- 전원 회로 안정성 확보를 위한 보호 회로 설계

주행 제어 시스템

STM32F303 MCU 기반 제어 시스템

Odometry: I2C 통신으로 센서 데이터 수집

- ADXL345 가속도 센서 활용
- QMC5883L 지자기 센서 활용
- ITG3205 자이로 센서 활용
- GY-85 센서 모듈 통합

Control : Pure_Pursuit

- TIM1~4 타이머 -> PWM, 엔코더
- TB6612FNG 모터 드라이버 사용
- 가변 Ld
- 구간 별 속도 제어 - 모터 FeedForward + PID

태양전지 전력 변환 시스템

태양전지의 전력을 효율적으로 변환하여 배터리를 충전하고, 이를 통해 MCU와 모터에 안정적인 전원을 공급하는 시스템

01

태양전지-Buck-배터리

- 태양전지에서 생성된 전력을
벅 컨버터로 변환
- 리튬이온 18650 배터리
3개 직렬 충전
- $4.2V \pm 5\%$ 출력 전압

02

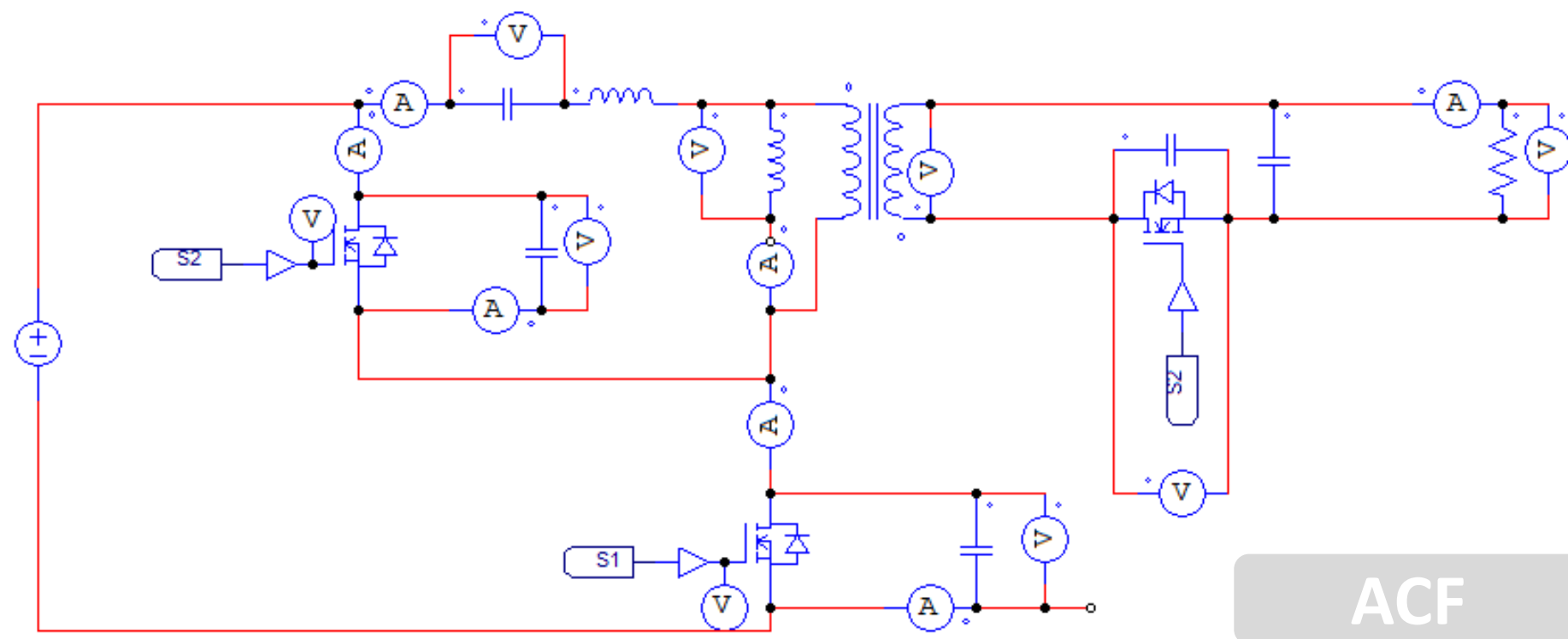
배터리-AC Flyback-MCU

- 배터리에서 공급되는
전력을 액티브 클램프 플라이백 컨버터
로 변환
- MCU 전원 공급
- $3.3V \pm 5\%$ 출력 전압

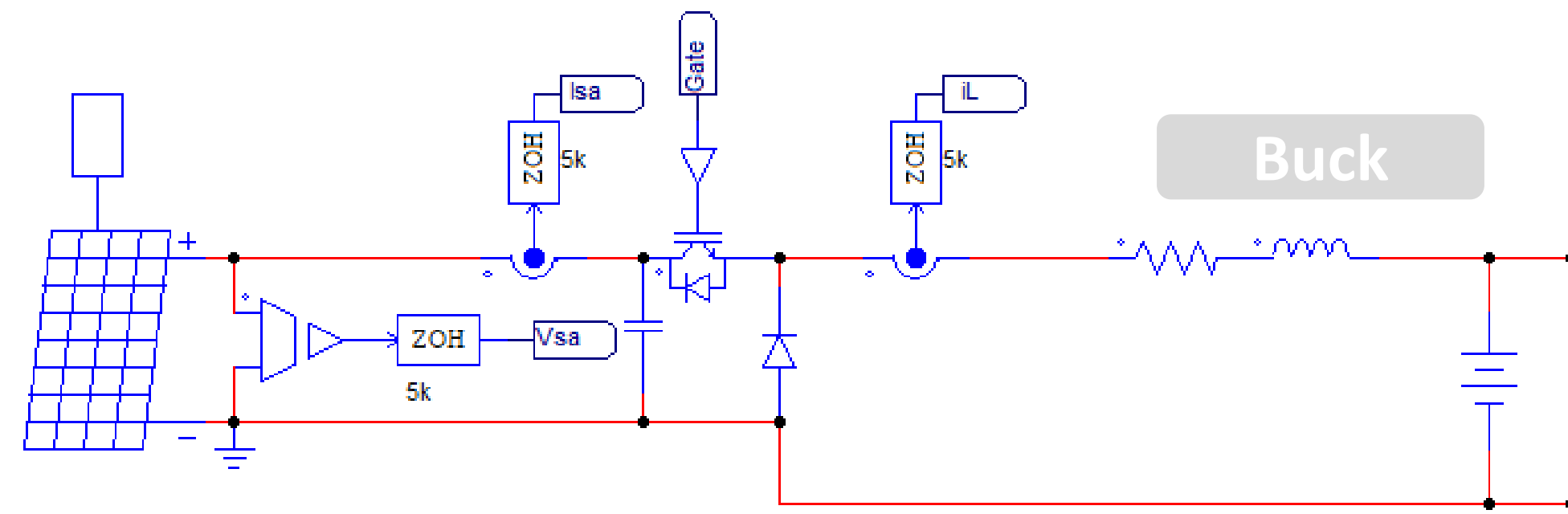
03

P&O 방식의 MPPT

- Perturb & Observe 방식
- 전압과 전류를 측정하여
전력 계산
- 최대 전력점 추적
- 듀티비 자동 조절



ACF



Buck

PSIM 액티브클램프플라이백 및 Buck 회로

PSIM P&O 알고리즘

```

void SimulationStep(
    double t, double delt, double *in, double *out,
    int *pnError, char * szErrorMsg,
    void ** reserved_UserData, int reserved_ThreadIndex, void * reserved_AppPtr)
{
    double Vsa, Isa, Psa;
    double DV = 0.5;
    int TUpdate = 3000;
    static int i = 0;

    static double Psa_old = 0.0;
    static double duty_old = 0.0;
    static double Verr_old = 0.0;
    static double Vsa_old=0;
    static double Vref=23.5;
    double duty, Verr;

    Vsa=in[0];
    Isa=in[1];

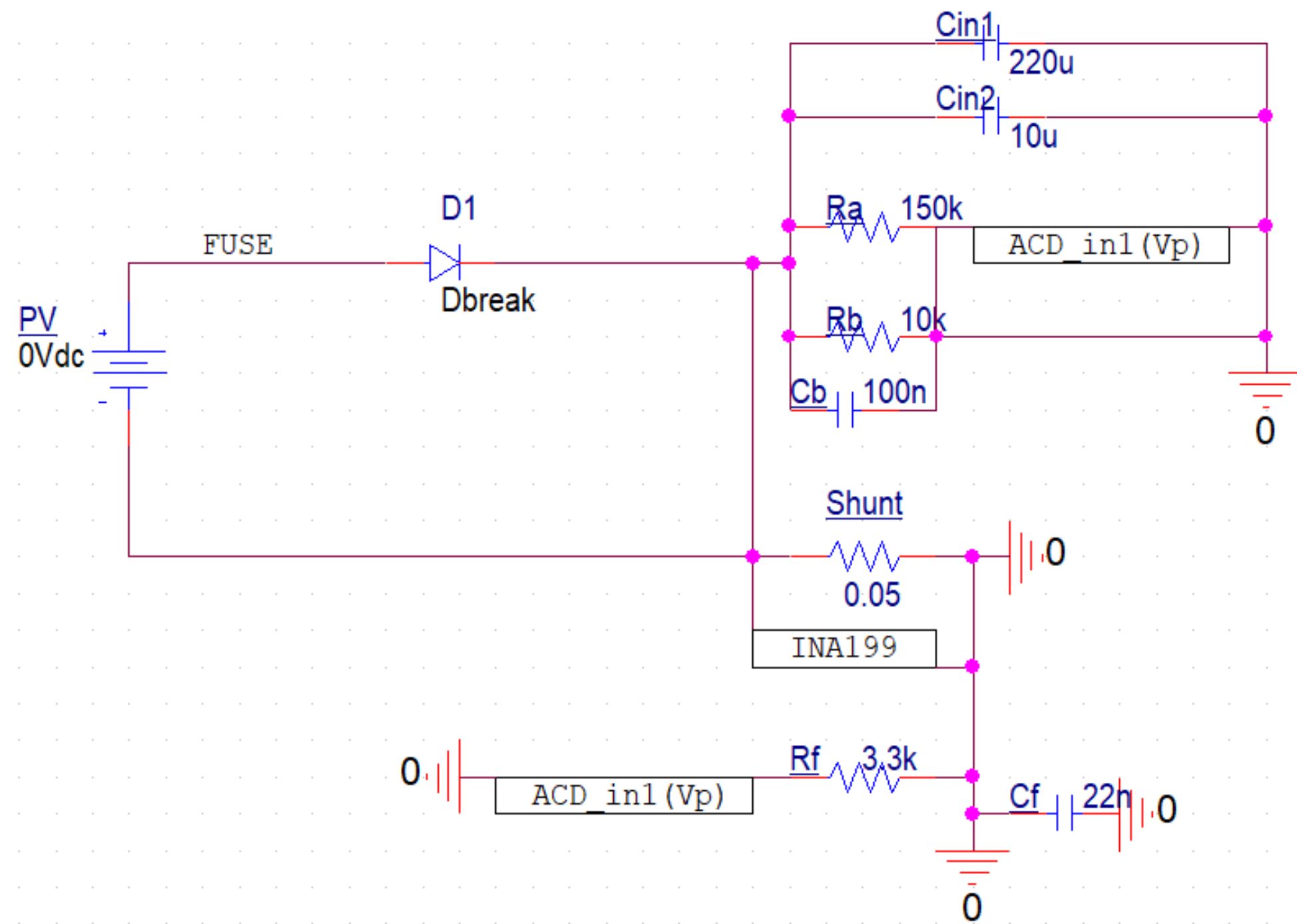
    i++;
    if(i%TUpdate == 0) {
        Psa = Vsa*Isa;
        if(Psa >= Psa_old) {
            if(Vsa>Vsa_old) Vref=Vref + DV;
            else Vref=Vref - DV;
        }
        else if(Psa <= Psa_old) {
            if(Vsa>Vsa_old) Vref=Vref - DV;
            else Vref=Vref + DV;
        }
        else if(Psa == Psa_old) {
            Vref = Vref;
        }
        Psa_old=Psa;
        Vsa_old=Vsa;
    }

    Verr=Vsa-Vref;;
    duty = duty_old + kp*Verr - kp*ki*Verr_old;
    duty_old=duty;
    Verr_old=Verr;
    out[0]=duty;
    out[1]=Vref;
}

```

PSIM C Block

OrCAD 센싱 회로



01 분압 저항

- $R_a = 150\text{k}\Omega$
- $R_b = 10\text{k}\Omega$
- 입력 전압(V_{in}) 분압용

02 분압 커패시터

- $C_b = 100\text{nF}$
- R_b 와 병렬 연결
- 노이즈 필터링

03 션트 저항

- $R_s = 0.05\text{ ohm}$
- 전류 측정용

04 전류 측정 증폭기

- INA180 또는 INA199 사용
- $R_f = 3.3\text{k}\Omega$, $C_f = 22\text{nF}$
- 차단 주파수 $f_c = 2.2\text{kHz}$

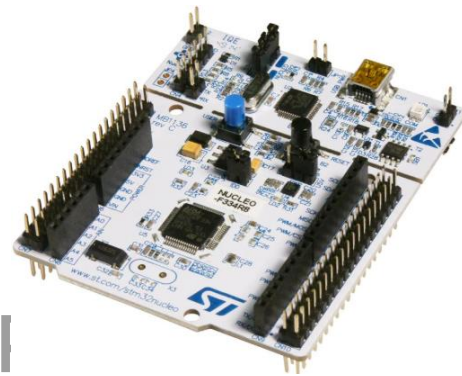
05 보호 회로

- 쇼트키 다이오드(SS54)
- TVS(25-30V)
- 퓨즈 1A
- ADC: AD7689 사용

3

설계 - 구동부 (1): 하드웨어 구성

MCU: STM32F303RE



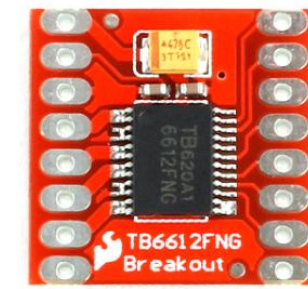
- Nucleo-64 개발 보드 사용
- Cortex-M4 ARM 32-bit CI
- 최대 클럭 72MHz
- SRAM 80KB, Flash 512KB
- FPU가 탑재되어 알고리즘 연산 중 이점

서보 모터



- MG996R 서보 모터
- 6V에서 0.17s/60deg
- PWM 신호: -90도(1000us)
- 0도(1500us), 90도(2000us)

모터 드라이버

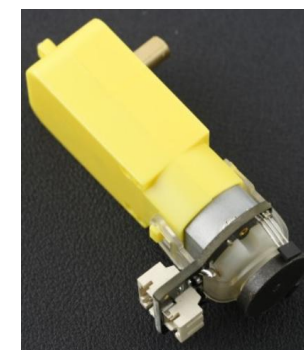


- TB6612FNG
 - 듀얼 모터 드라이버
 - PWM
- PCLK 72MHz, PSC:3, ARR: 999

운영체제: FreeRTOS

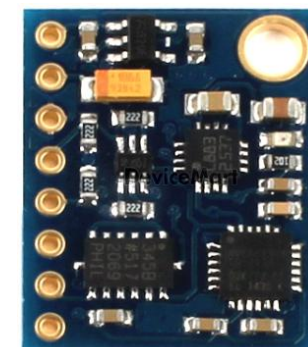
- CMSIS-V2 기반 FreeRTOS 사용
- 센서 데이터 수집 병렬화
- 오도메트리 연산 병렬화
- 제어 체계 병렬화

DC 모터 & 엔코더



- 6V 160rpm DC 모터
 - 120:1 기어비
 - TB6612FNG 드라이버
 - 960 펄스/회전 엔코더
- (Encoder mode T1 & T2 : 3840)

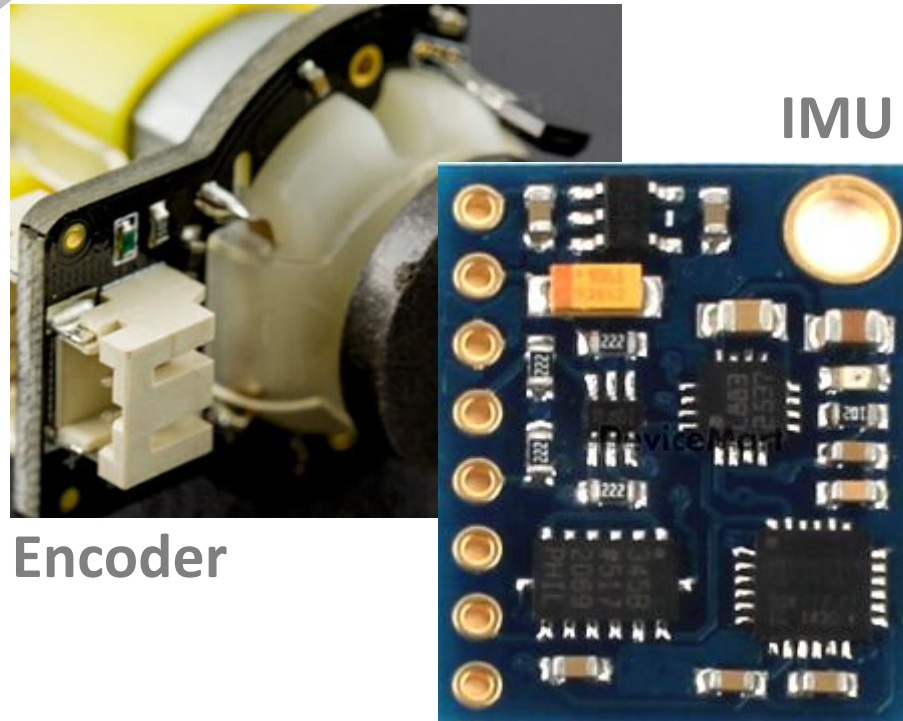
센서 구성



- GY-85
 - ADXL345 가속도 센서
 - ITG3205 자이로스코프
 - QMC5883L 지자기 센서
- Not LiDAR & Camera ?

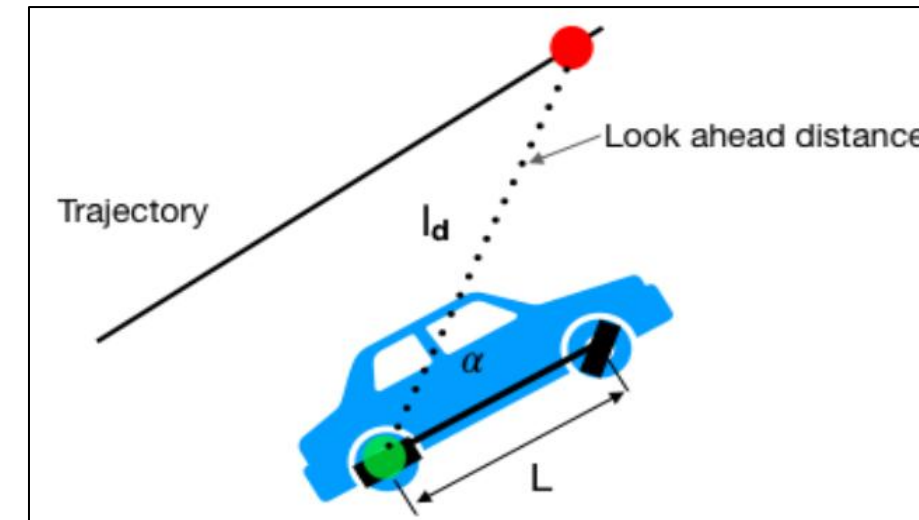
$$16,000 + 4,200 + 4,200 + 2 \times 10,000 + 7,000 + 4 \times 800 + 12,000 + 2,500 = 69,100 \text{ 원}$$

MCU	서보	모터 드라이버	DC+엔코더	GY-85	바퀴	차체 판	배송비
-----	----	------------	--------	-------	----	------	-----



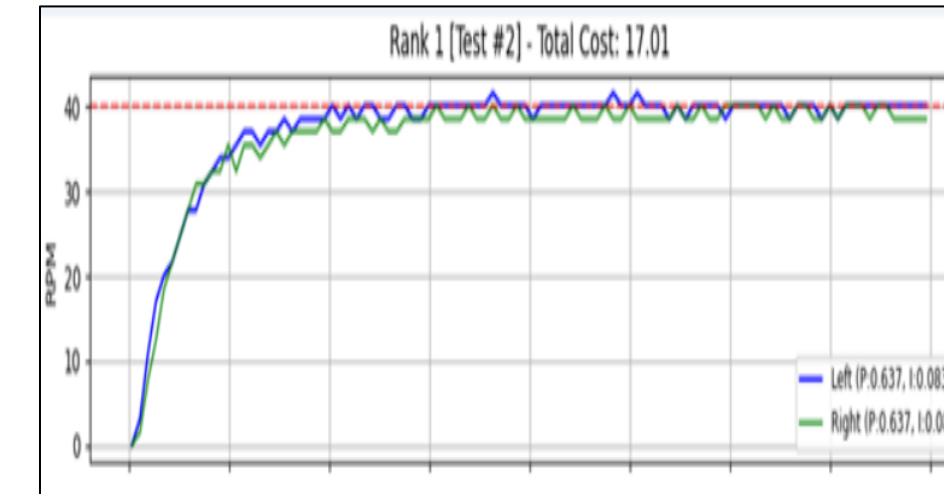
Task 1: Sensor

- 10ms 주기로 실행
- 가속도, 자이로, 지자기 데이터 수집
- 엔코더 데이터 수집
- 센서 퓨전(상보 필터) 적용
- 직진 주행 거리 및 Heading 측정



Task 2: Navigate

- 50ms 주기로 실행
- Pure-Pursuit 알고리즘 구동
- 현재 위치 기반 경로 추적
- 서보 모터 지령 생성
- 주행 구간에 따른 DC 모터 지령 생성



Task 3: Control

- 10ms 주기로 실행
- DC 모터 PID 제어 수행
- P, I 게인 적용
- Feed-Forward 적용



Task 4: LCD

- 500ms 주기로 실행
- 주행 거리 표시
- 현재 RPM 표시
- 현재 주행 Waypoint 표시

공유 데이터 관리 - Mutex Protected

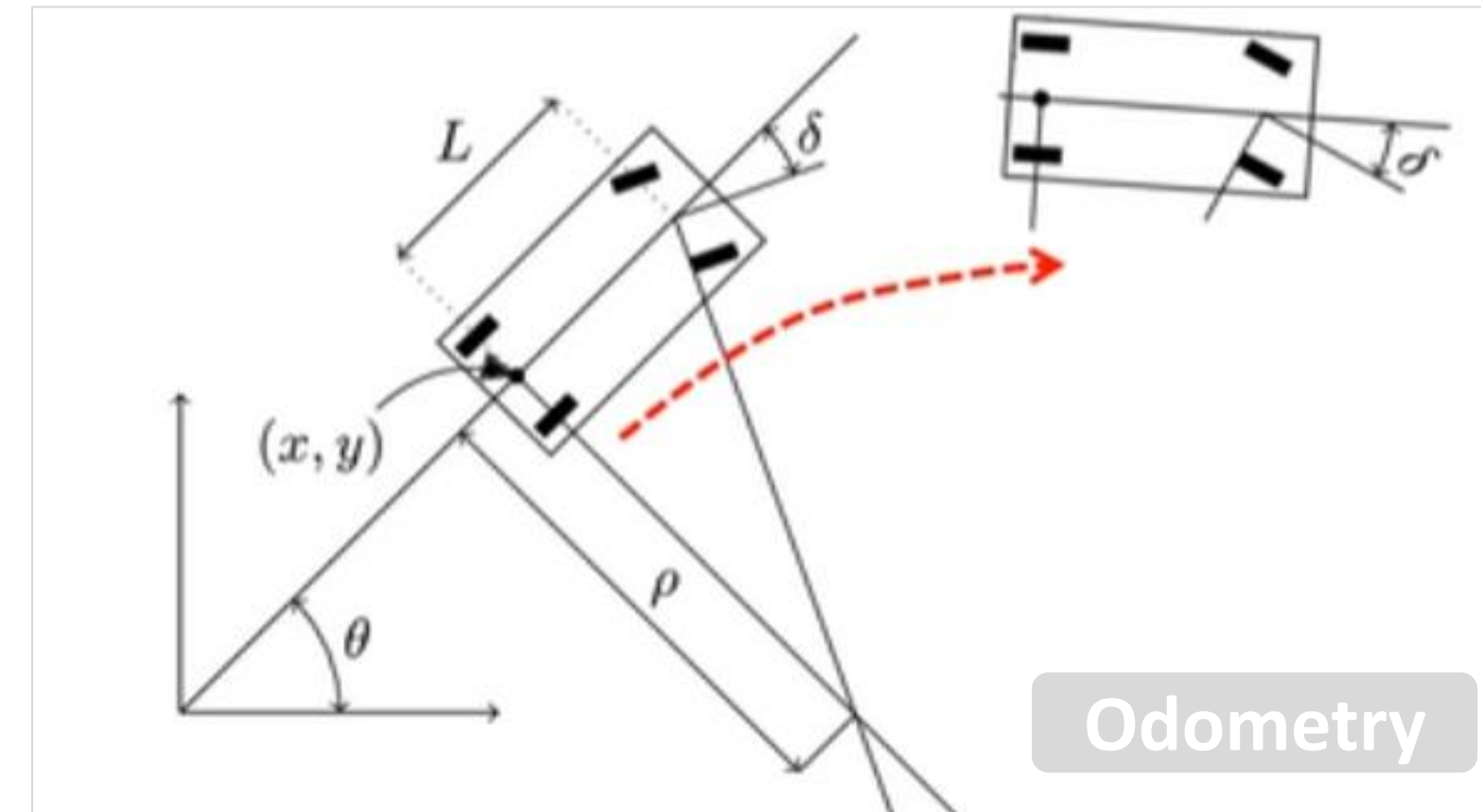
-> g_RobotData : 현 상태와 지령을 저장하는 전역 변수

3

센서 특성 및 한계

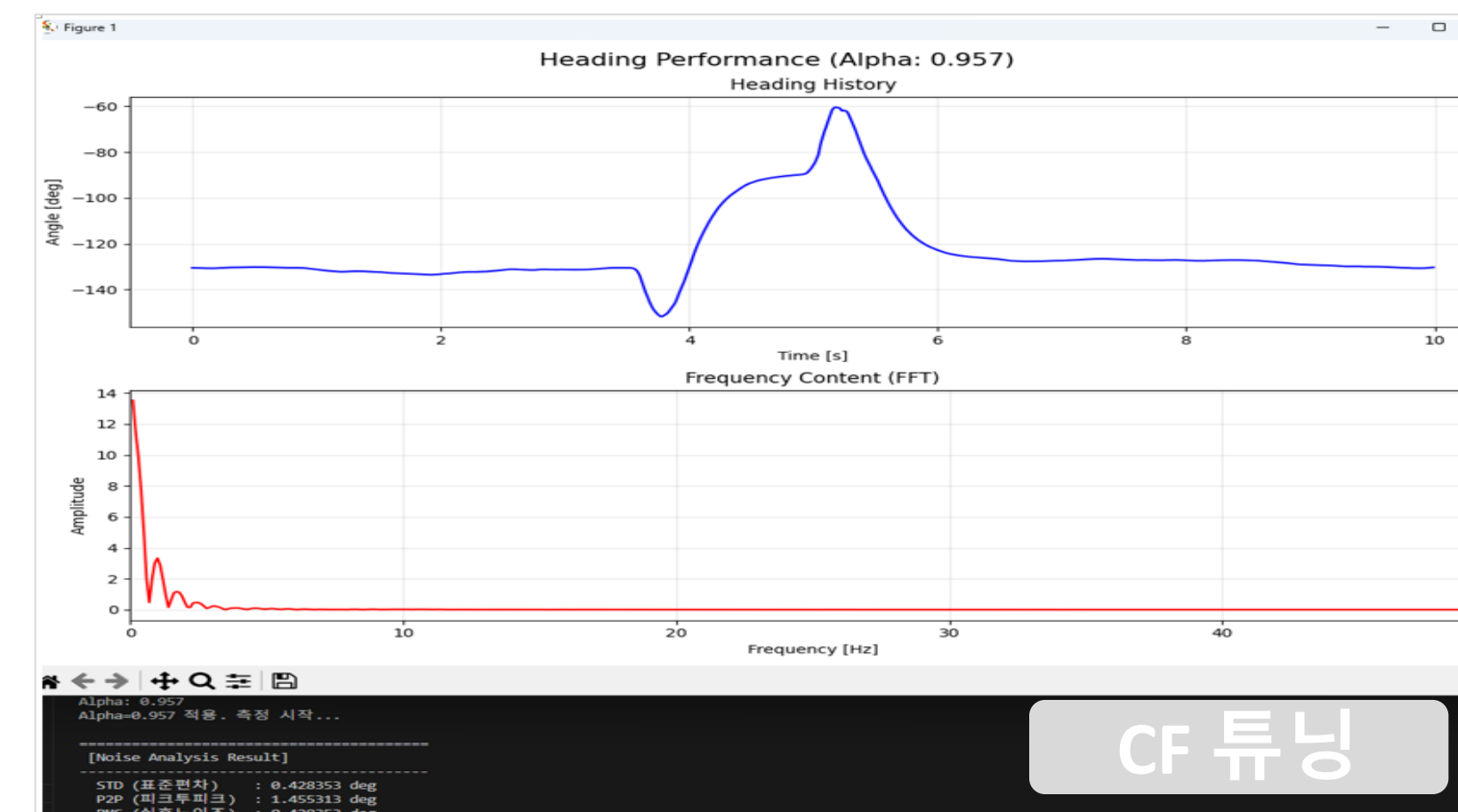
- 자이로: 단기 추종 속도 빠르고 정확하나 드리프트 발생
- 지자기: 주변 자계에 의한 오차 및 노이즈 극심
- 가속도: 타 센서의 기울어짐 보정 가능

1↓



상보 필터 융합

- 자이로와 지자기 센서 데이터의 주파수 분석을 통한 최적 필터 설계
- 1차 EMA(Exponential Moving Average) 상보필터 적용
- 드리프트 제거 및 노이즈 감소 효과
- Python을 활용한 상보 필터 설계



차량 구동 제어 시스템

01

DC 모터 제어

- Feed-Forward + PID
제어기 적용

02

PID 튜닝

- HILS(Hardware In
the Loop Simulation)
- 자동화된 PID
파라미터 최적화

03

서보모터 제어

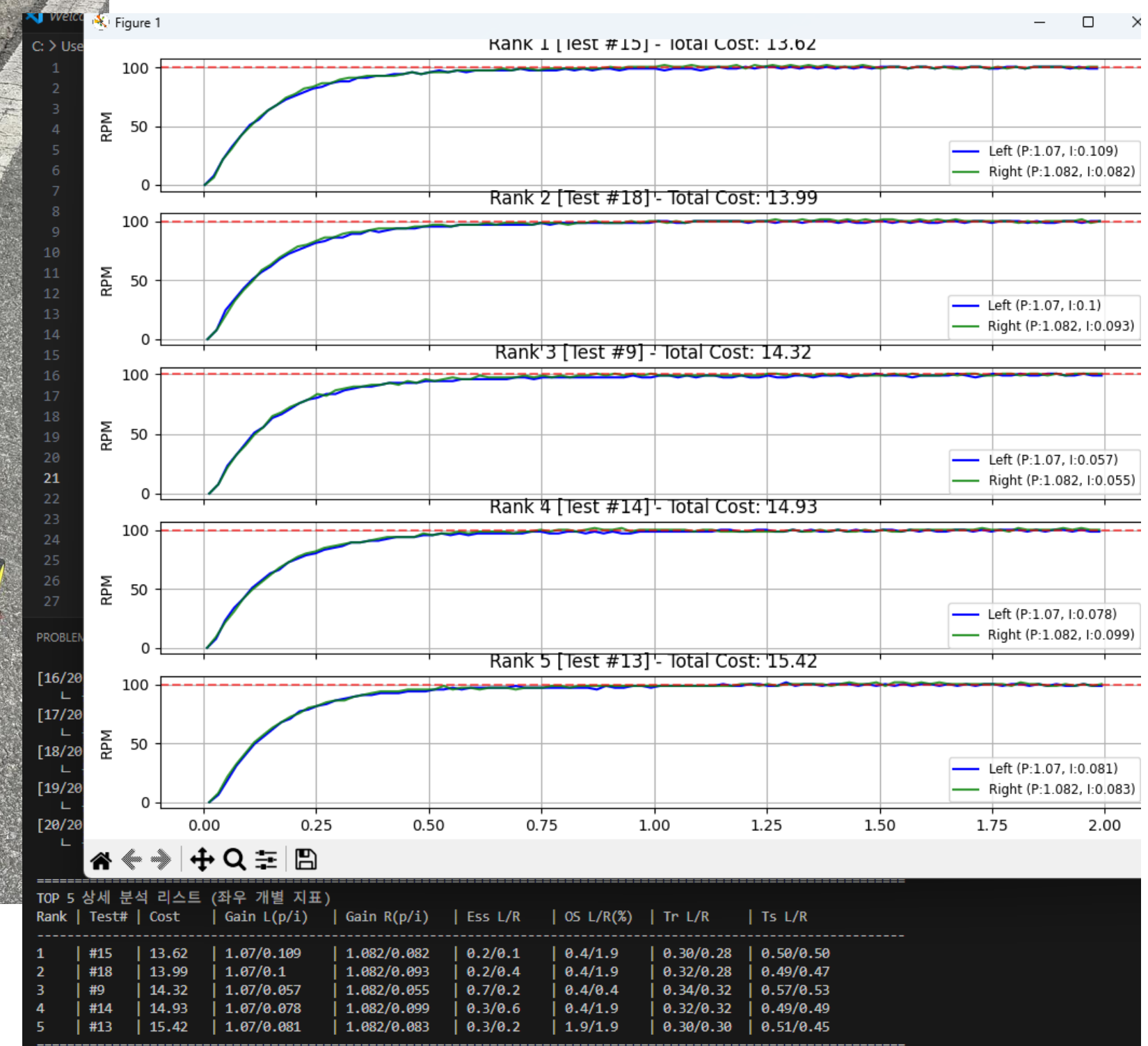
- Open-Loop 제어
방식 채택

02

PID 튜닝

- HILS(Hardware In the Loop Simulation) 모사 방식 활용
- Python 프로그램과 MCU 연결(UART)
- 자동화된 PID 파라미터 최적화

차량 구동 제어 시스템



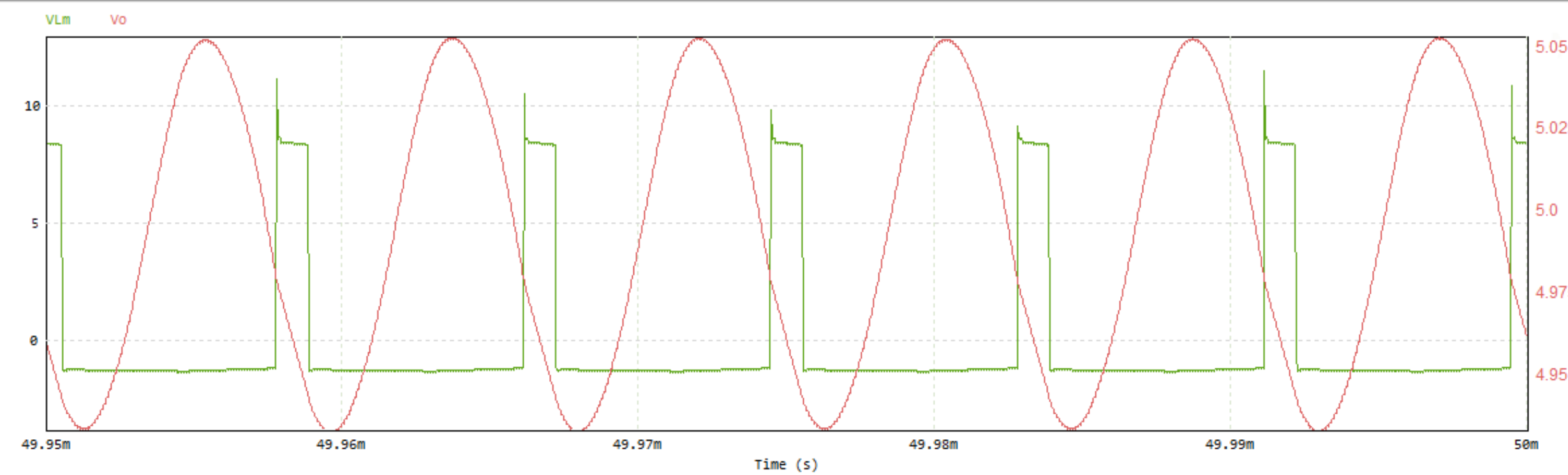
저속
중속
고속

구간별 Kp, Ki 적용

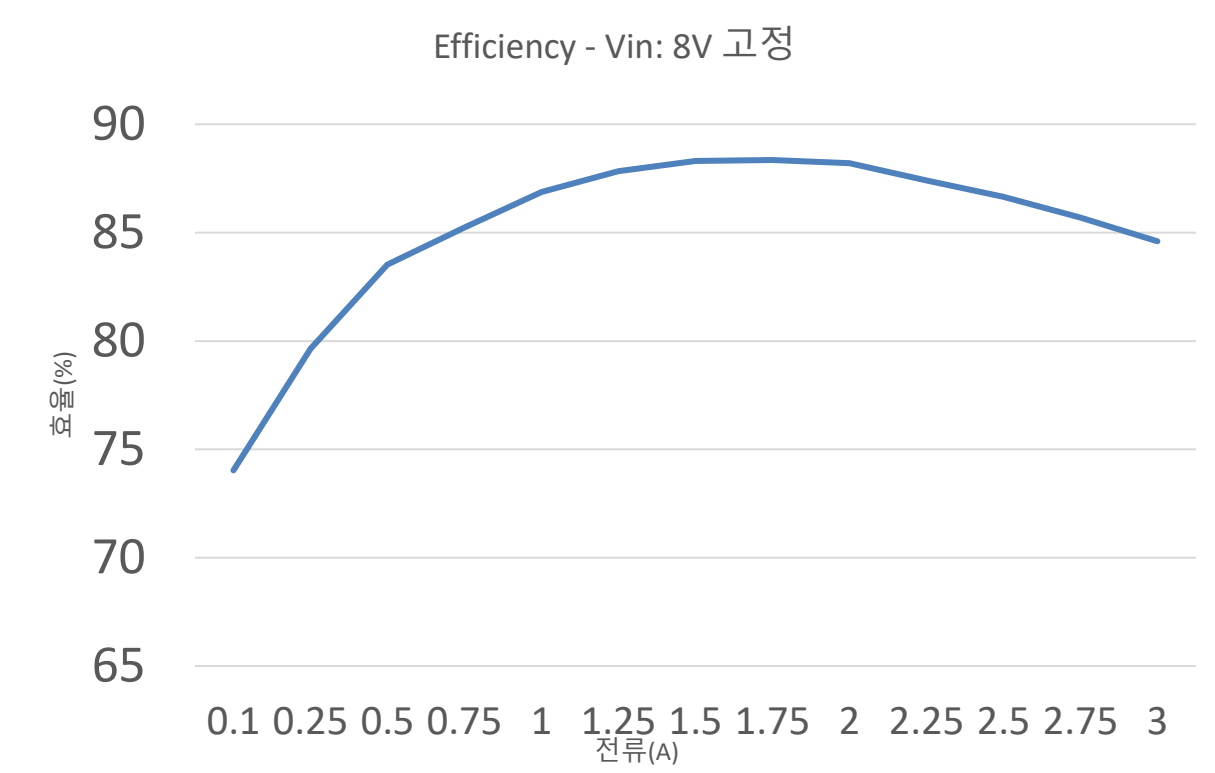
```
// 히스테리시스 임계값
#define LOW_TO_MID_UP 55.0f
#define MID_TO_LOW_DOWN 50.0f

#define MID_TO_HIGH_UP 77.0f
#define HIGH_TO_MID_DOWN 73.0f
```

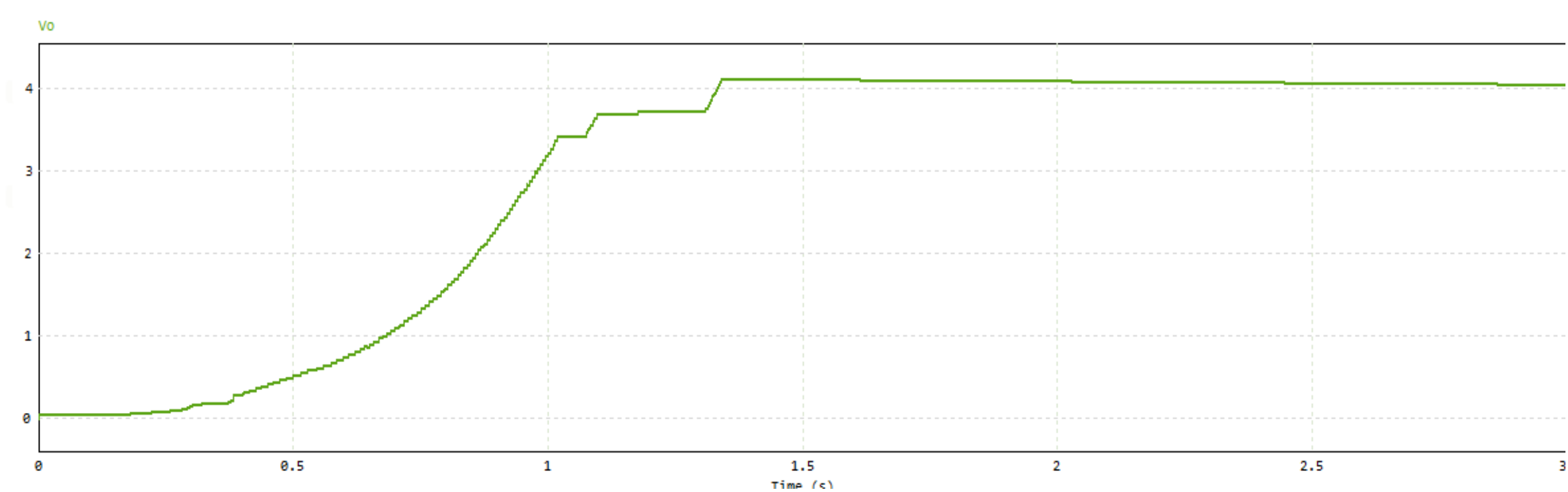

Active Clamp Flyback 컨버터



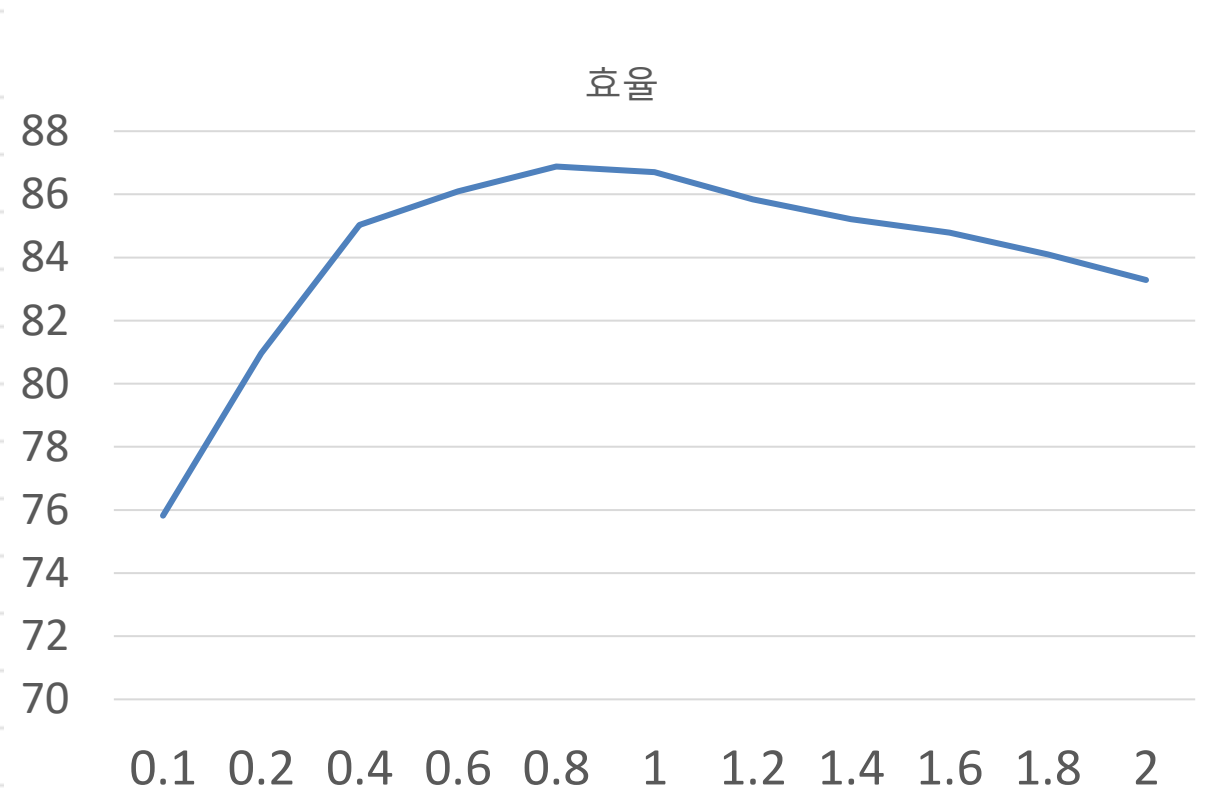
Load_Curr ent(A)	Vin_Batt(V)	lin_Batt(A)	Vout_Meas (V)	P_in(W)	P_out(W)	Efficiency(%)	Ripple_Vpp (mV)
0.1	3.7	0.118	3.31	0.437	0.331	75.82	12.5
0.2	3.7	0.221	3.31	0.818	0.662	80.96	14.2
0.4	3.69	0.422	3.31	1.557	1.324	85.03	18.5
0.6	3.68	0.625	3.3	2.3	1.98	86.09	22.8
0.8	3.67	0.828	3.3	3.039	2.64	86.88	26.4
1	3.66	1.04	3.3	3.806	3.3	86.7	31.5
1.2	3.65	1.26	3.29	4.599	3.948	85.84	38.2
1.4	3.64	1.485	3.29	5.405	4.606	85.21	45.1
1.6	3.62	1.715	3.29	6.208	5.264	84.79	52.6
1.8	3.6	1.95	3.28	7.02	5.904	84.1	60.4
2	3.58	2.2	3.28	7.876	6.56	83.29	69.8



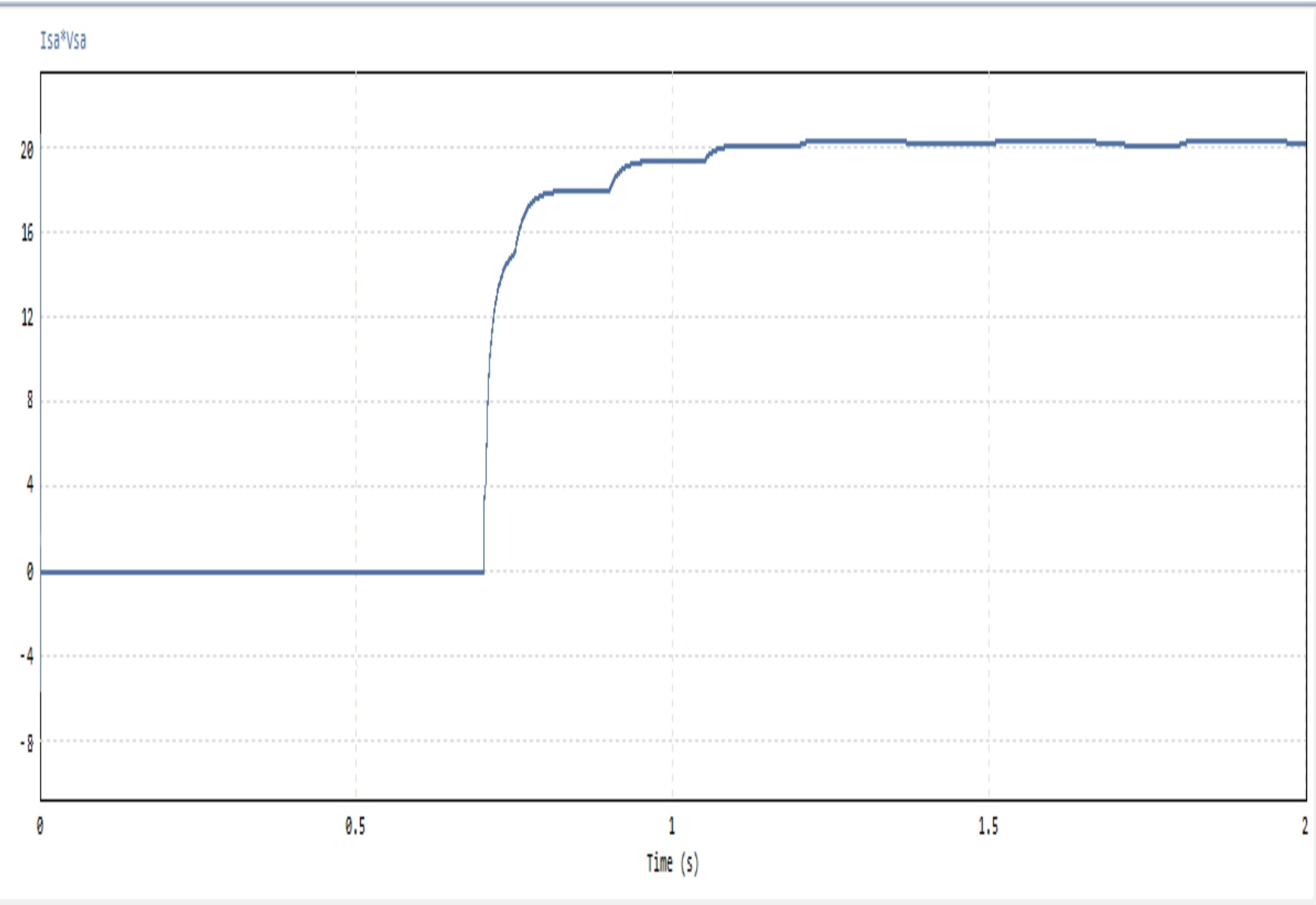
Buck 컨버터



Load_Curr ent(A)	Vin_Meas(V)	lin_Meas(A)	Vout_Meas(V)	P_in(W)	P_out(W)	Efficiency(%)	Ripple_Vpp (mV)
0.1	8.01	0.071	4.21	0.569	0.421	74.03	18.5
0.25	8.01	0.165	4.21	1.322	1.053	79.64	22.1
0.5	8	0.315	4.21	2.52	2.105	83.53	28.4
0.75	8	0.462	4.2	3.696	3.15	85.23	34.2
1	7.99	0.605	4.2	4.834	4.2	86.88	41.5
1.25	7.99	0.748	4.2	5.977	5.25	87.84	48.9
1.5	7.98	0.892	4.19	7.118	6.285	88.3	58.2
1.75	7.98	1.04	4.19	8.299	7.333	88.35	68.5
2	7.97	1.192	4.19	9.5	8.38	88.21	79.1
2.25	7.97	1.35	4.18	10.76	9.405	87.41	88.4
2.5	7.96	1.515	4.18	12.059	10.45	86.65	98.2
2.75	7.96	1.685	4.18	13.413	11.495	85.7	108.5
3	7.95	1.86	4.17	14.787	12.51	84.6	121.3



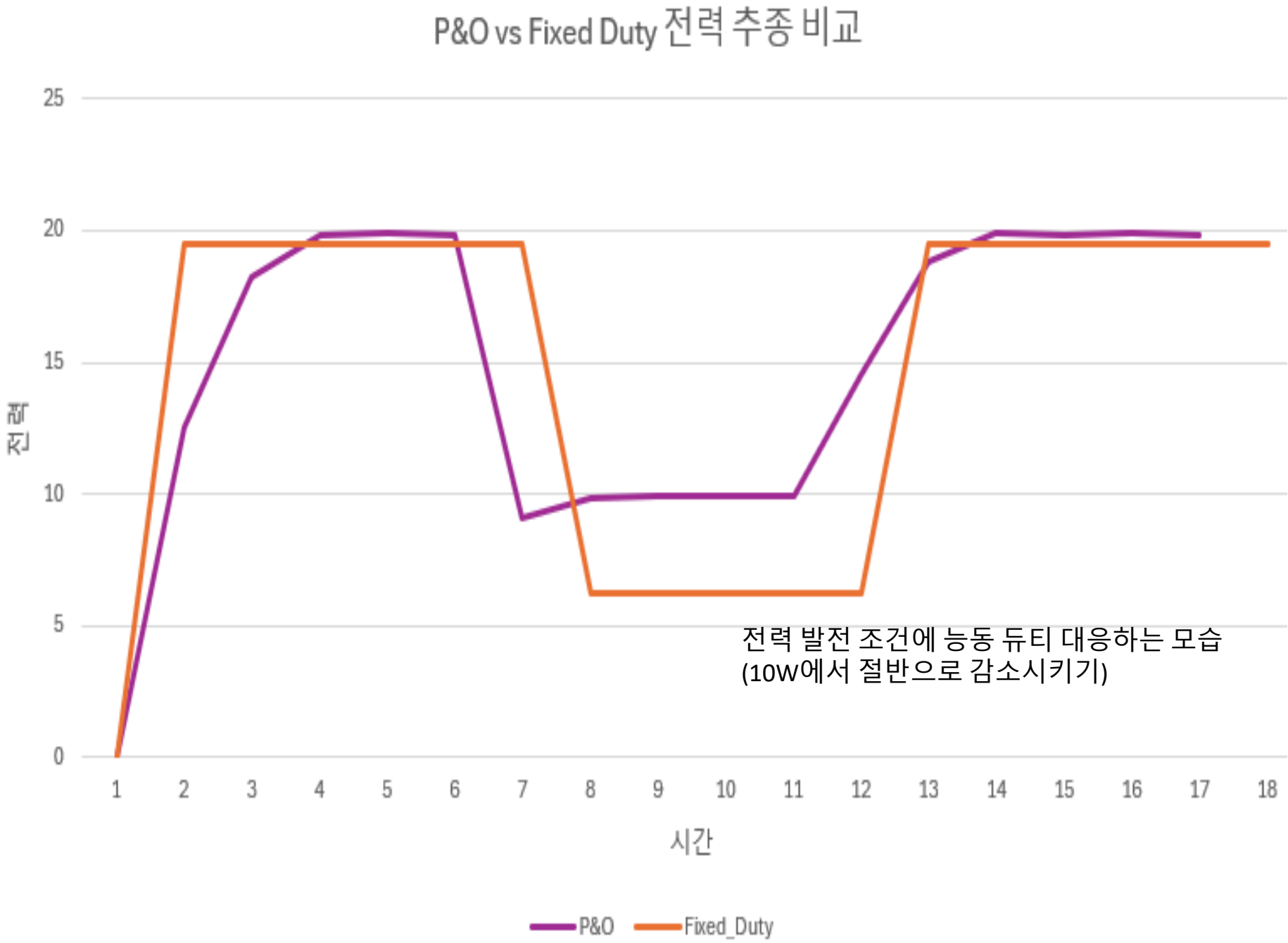
MPPT 동작(입력 전력)

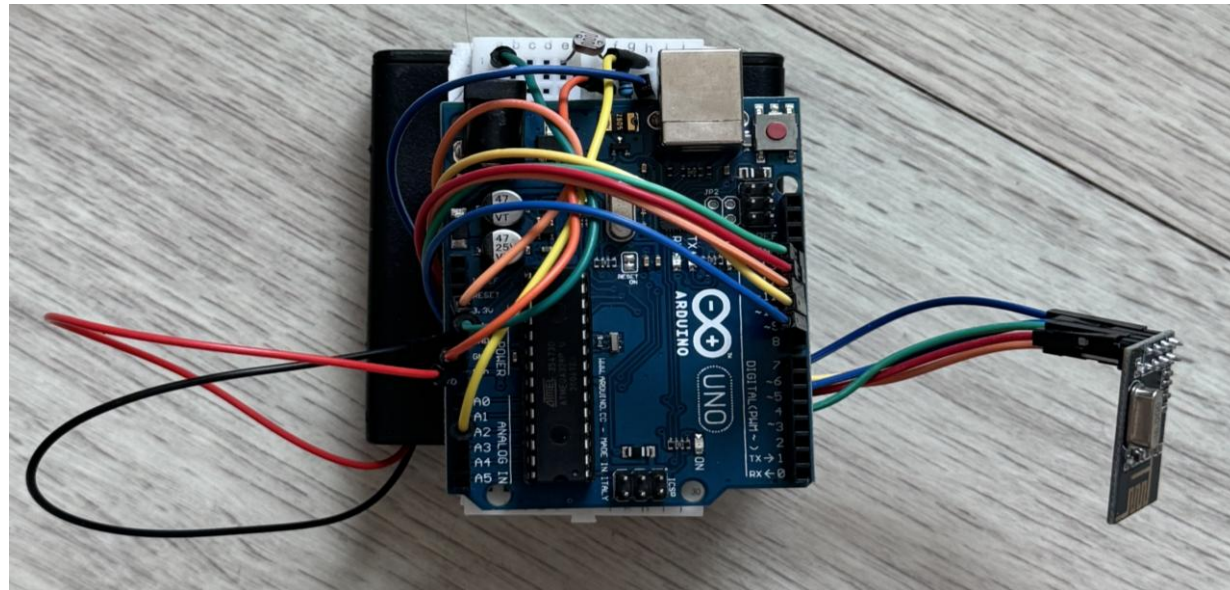


전력 변화 유도 테스트 로그

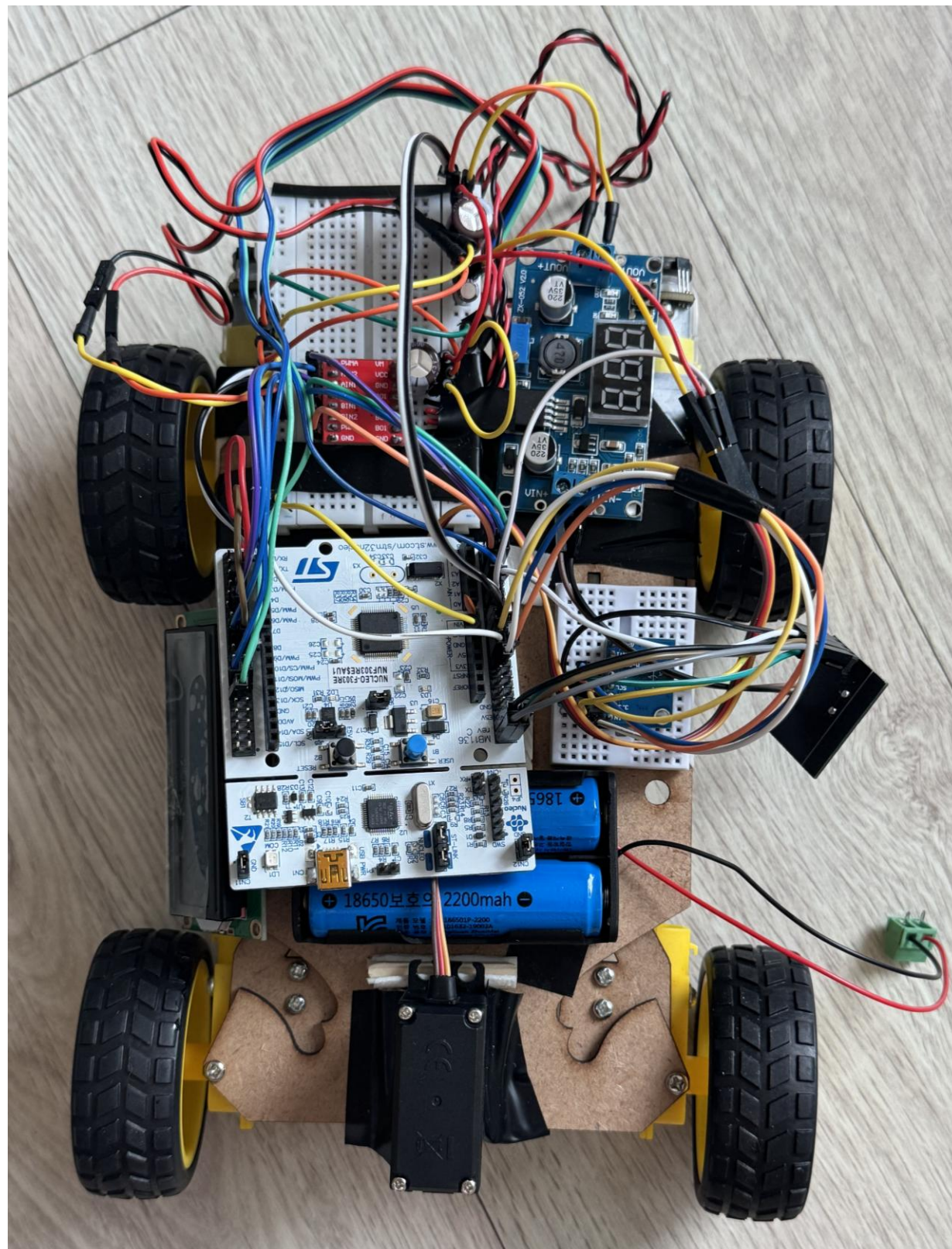
Time	Fixed_Duty	P&O	Duty_Cycle
0	19.5	0	0
0.5	19.5	12.5	20
1	19.5	18.2	35
1.5	19.5	19.85	44.5
2	19.5	19.95	45.2
2.5	19.5	19.85	44.8
3	6.2	9.1	44.8
3.5	6.2	9.85	38.5
4	6.2	9.95	35.2
4.5	6.2	9.88	34.8
5	6.2	9.92	35.5
5.5	19.5	14.5	35.5
6	19.5	18.8	40.2
6.5	19.5	19.92	44.9
7	19.5	19.82	45.5
7.5	19.5	19.94	45
8	19.5	19.85	44.5

전력량 비교

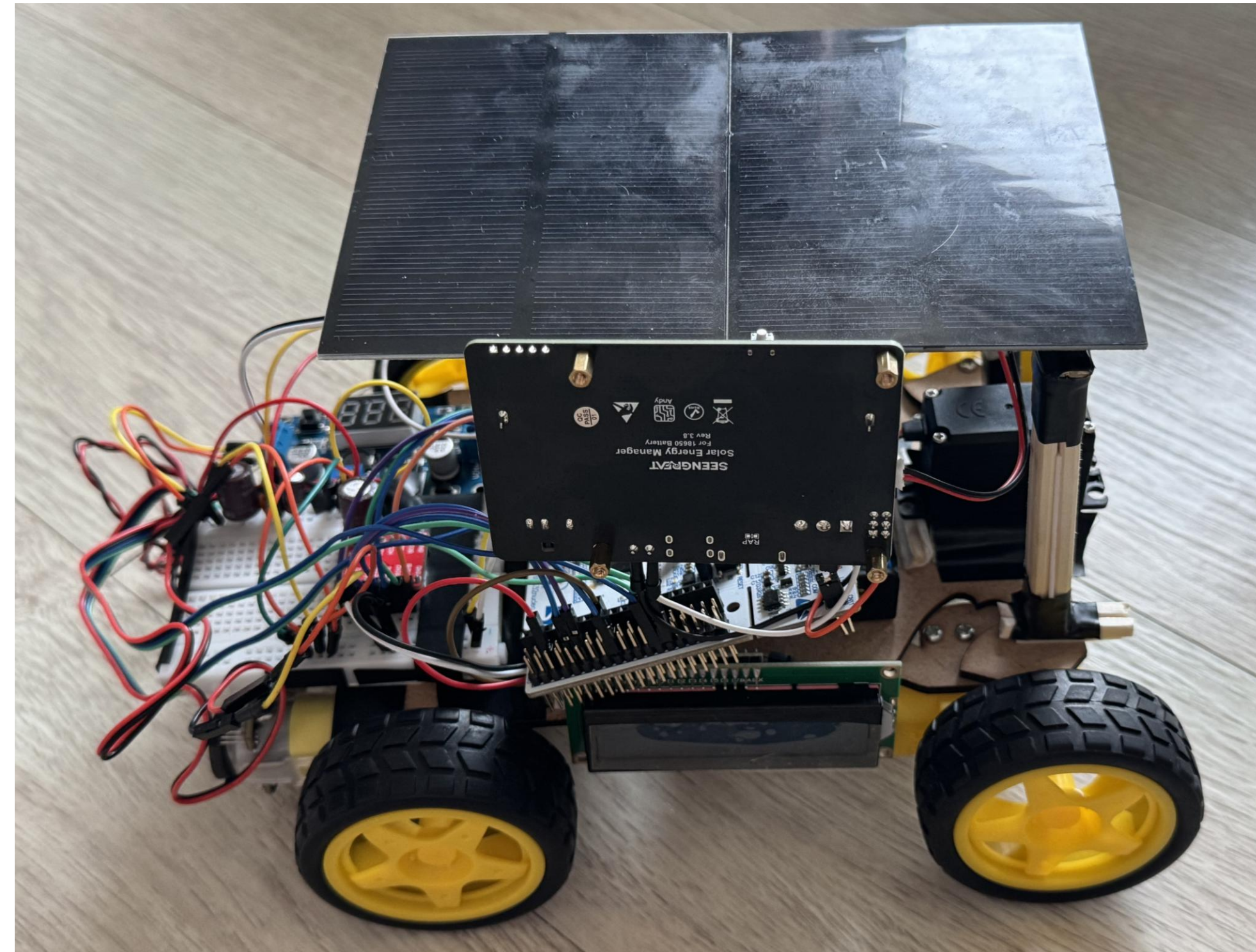




주차장 노드 조도 센서 + RF 모듈



미니카 결과물



태양전지 + MPPT 부착

nRF 무선 통신 결과

```
nRF | 아두이노 1.8.19 (Windows Store 1.8.57.0)
파일 편집 스케치 툴 도움말

nRF $

#include <SPI.h>
#include <nRF24L01.h>
#include <RF24.h>

RF24 radio(7, 8);

// 맨 앞에 0x010이 번호임
const byte address[6] = {0x01, 0xC2, 0xC2, 0xC2, 0xC2};

struct __attribute__((packed)) Waypoint_t {
    float x;
    float y;
    float target_speed;
    uint8_t is_stop_point;
};

Waypoint_t my_path[5] = {
    {0.0, 0.0, 50.0, 0},
    {2.0, 1.5, 60.0, 0},
    {3.0, 2.0, 70.0, 0},
    {4.0, 1.0, 60.0, 0},
    {5.0, 0.0, 0.0, 1}
};
```

STM32CubeIDE – Debugging: Live Expression

g_rx_complete_flag	uint8_t [6]	[6]
g_rx_complete_flag[0]	uint8_t	0 'W0'
g_rx_complete_flag[1]	uint8_t	0 'W0'
g_rx_complete_flag[2]	uint8_t	0 'W0'
g_rx_complete_flag[3]	uint8_t	0 'W0'
g_rx_complete_flag[4]	uint8_t	0 'W0'
g_rx_complete_flag[5]	uint8_t	0 'W0'
g_node_data	TotalData_t [6]	[6]
my_path	Waypoint_t [5]	[5]
my_path[0]	Waypoint_t	{...}
x	float	0
y	float	0
target_speed	float	0
is_stop_point	uint8_t	0 'W0'
my_path[1]	Waypoint_t	{...}
x	float	0
y	float	0
target_speed	float	0
is_stop_point	uint8_t	0 'W0'
my_path[2]	Waypoint_t	{...}
x	float	0
y	float	0
target_speed	float	0
is_stop_point	uint8_t	0 'W0'
my_path[3]	Waypoint_t	{...}
x	float	0
y	float	0
target_speed	float	0
is_stop_point	uint8_t	0 'W0'
my_path[4]	Waypoint_t	{...}
x	float	0
y	float	0
target_speed	float	0
is_stop_point	uint8_t	0 'W0'
current_cds_value	uint16_t	0

송신 데이터

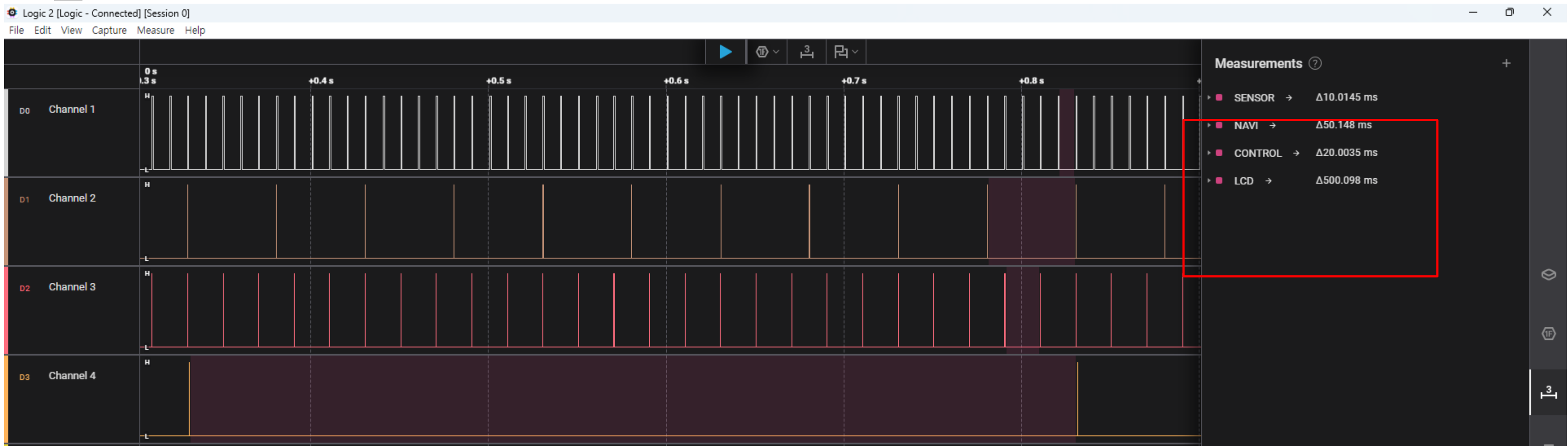
수신 전

수신 후

g_rx_complete_flag	uint8_t [6]	[6]
g_rx_complete_flag[0]	uint8_t	0 'W0'
g_rx_complete_flag[1]	uint8_t	1 'W001'
g_rx_complete_flag[2]	uint8_t	0 'W0'
g_rx_complete_flag[3]	uint8_t	0 'W0'
g_rx_complete_flag[4]	uint8_t	0 'W0'
g_rx_complete_flag[5]	uint8_t	0 'W0'
g_node_data	TotalData_t [6]	[6]
my_path	Waypoint_t [5]	[5]
my_path[0]	Waypoint_t	{...}
x	float	0
y	float	0
target_speed	float	50
is_stop_point	uint8_t	0 'W0'
my_path[1]	Waypoint_t	{...}
x	float	2
y	float	1.5
target_speed	float	60
is_stop_point	uint8_t	0 'W0'
my_path[2]	Waypoint_t	{...}
x	float	3
y	float	2
target_speed	float	70
is_stop_point	uint8_t	0 'W0'
my_path[3]	Waypoint_t	{...}
x	float	4
y	float	1
target_speed	float	60
is_stop_point	uint8_t	0 'W0'
my_path[4]	Waypoint_t	{...}
x	float	5
y	float	0
target_speed	float	0
is_stop_point	uint8_t	1 'W001'
current_cds_value	uint16_t	56

4

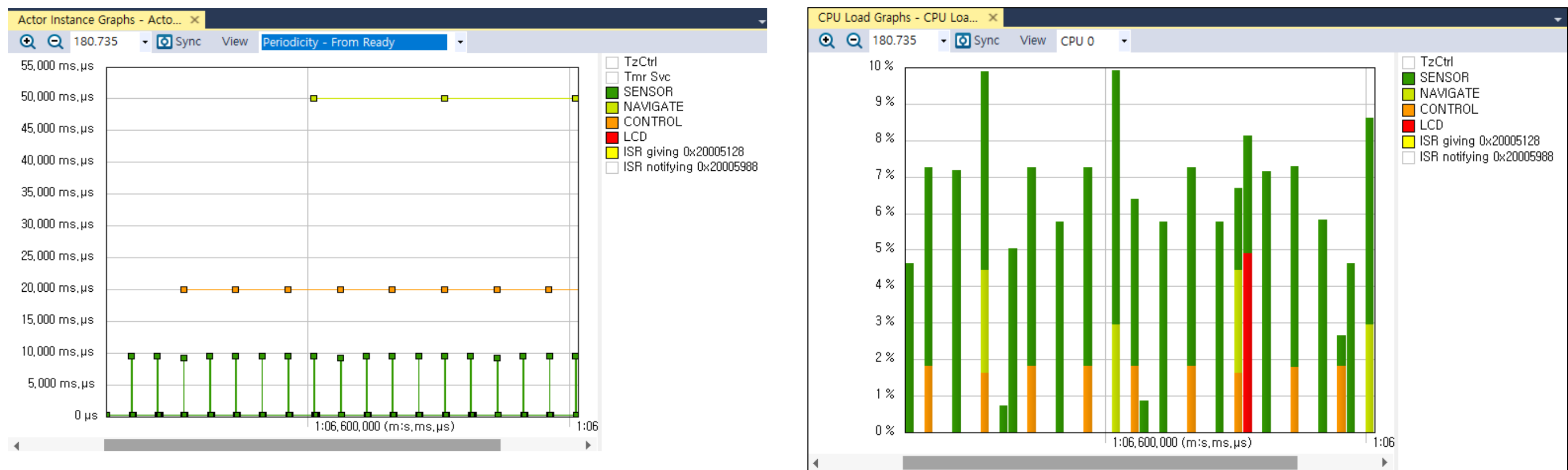
Verification of Periodicity – using Logic Analyzer



지터 없이
일정한 주기가 지켜짐을 확인

(Write GPIO Pin을 삽입해 확인)

Verification of Periodicity – using Tracealyzer



(참고) CubeIDE – FreeRTOS TASK List

이름	우선순위 (Bas...	스택의 시작	스택의 끝	상태	이벤트 객체	최소로 사용 가능한 스택	Run Time (%)
CONTROL	40/40	0x20005e60	0x20006534 ...	지연된		>256	48%
IDLE	0/0	0x200047d0	0x2000497c ...	실행		>256	CTR OVR
LCD	8/8	0x200066d0	0x2000699c ...	지연된		>256	6%
NAVIGATE	24/24	0x200059f0	0x20005cb4 ...	지연된		>256	35%
SENSOR	48/48	0x20005180	0x20005864 ...	지연된		>256	CTR OVR
Tmr Svc	2/2	0x20004370	0x200046fc <...	블럭된	TmrQ	>256	<1%
TzCtrl	1/1	0x2000050c	0x200008b4 ...	지연된		>256	CTR OVR

(참고) Heap 상태 – 동적할당 쓰지 않았음

myFreeHeap	size_t	5512
myMinHeap	size_t	5512

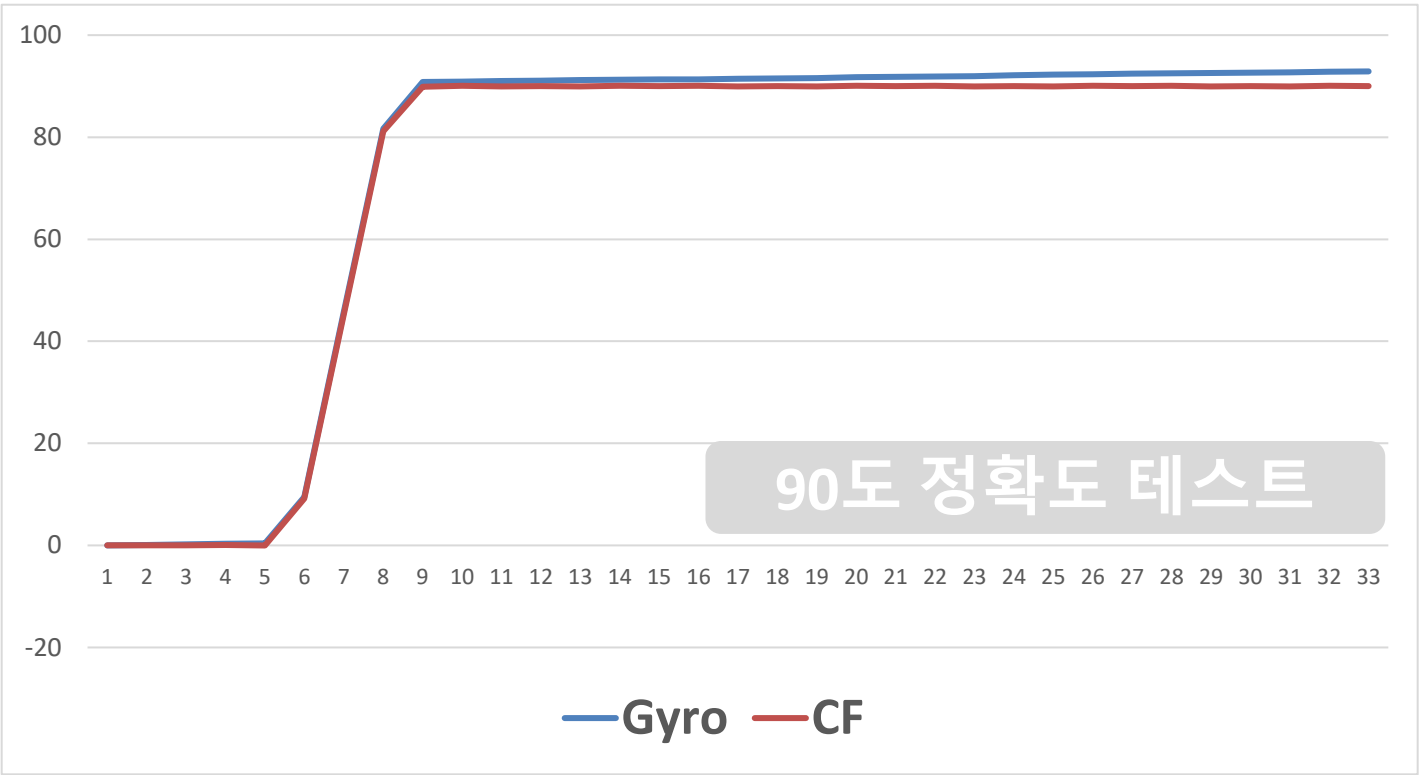
실측 기반 보정 / 튜닝

```
#define WHEELBASE_M      0.15f    // 미니카 축간거리 (15cm)
#define DIAMETER_M       0.065f   // 데이터 시트상 지름
#define CIRCUMFERENCE_M (3.1415926f * DIAMETER_M) * 1.01335375053
#define COUNTS_PER_REV_R 3863.0f  // 엔코더 R
#define COUNTS_PER_REV_L 3871.0f  // 엔코더 L

#define RPM_MIN_LD      40.0f     // 이 속도 이하일 때 최소 Ld
#define RPM_MAX_LD      80.0f     // 이 속도 이상일 때 최대 Ld
#define LD_MIN          0.35f     // 최소 록어헤드 (코너링)
#define LD_MAX          0.8f      // 최대 록어헤드 (직진 주행)
#define WP_PASS_RADIUS  0.2f      // 이 거리 안이면 다음 경유지로 넘어감
#define WP_STOP_RADIUS  0.03f     // 정지거리 3cm
#define STEER_GAIN       1.0f      // 조향 약하게 하고 싶을 이거 줄이면댐
#define MAX_STEER_DEG   30.0f     // 최대 조향각 제한 (도)
#define BRAKING_DIST     0.3f     // 40cm 전부터 다음 목표 속도로 감속 시작
#define FINAL_STOP_DIST  0.2f     // 마지막 정지 지점 30cm 전부터 0으로 감속
```



Heading 성능

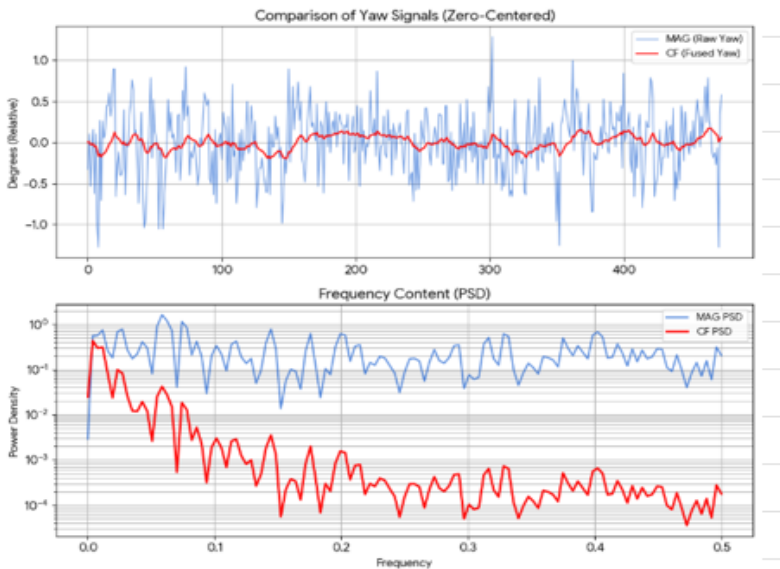


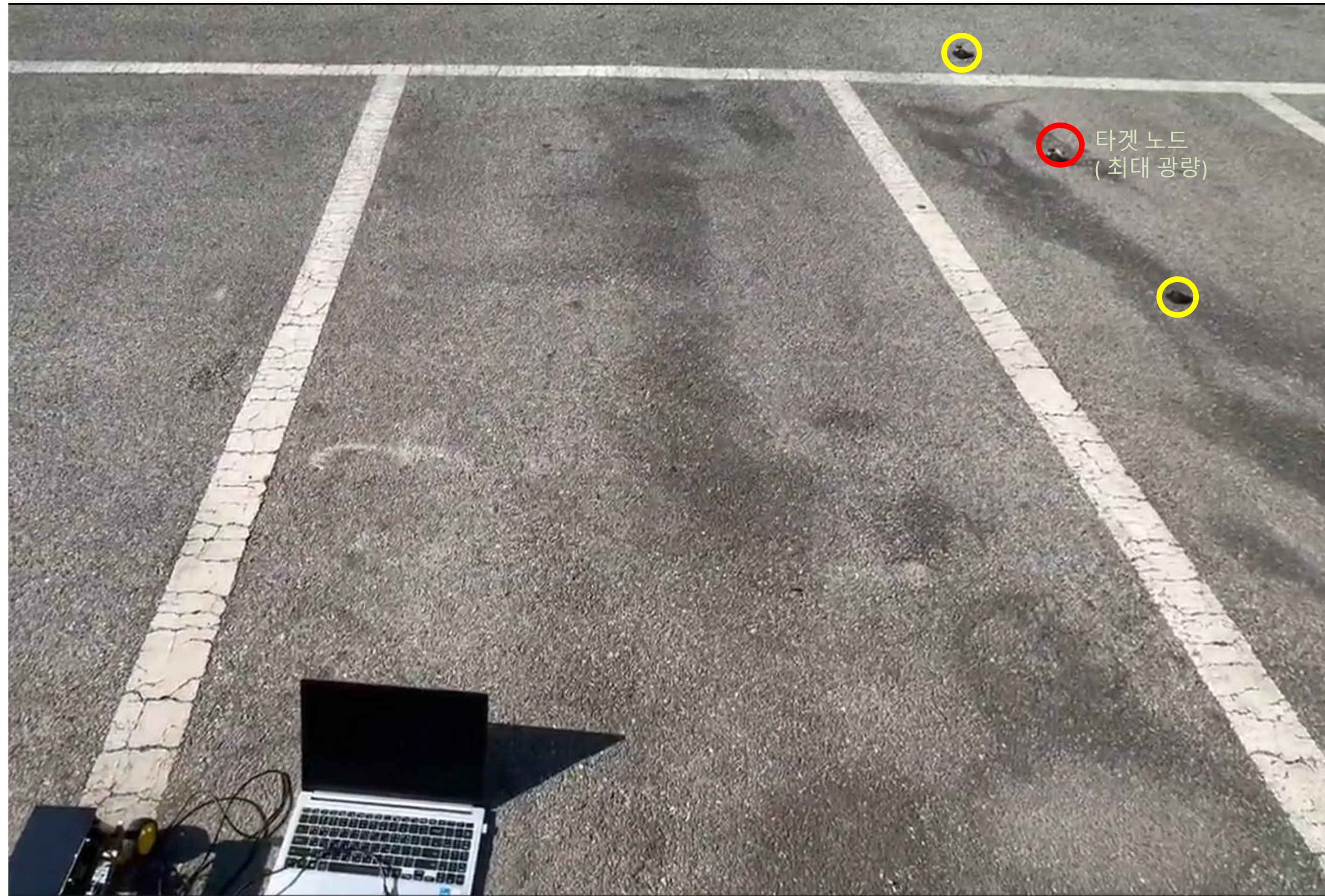
주행 오차 테스트

자 측정	X[m]	Y[m]	오차[cm]	오차[%]	3cm오차[%]
1회차	2.976	2.984	2.88	0.679	11.56
2회차	2.977	2.982	2.92	0.688	7.94
3회차	2.975	2.984	2.97	0.7	3.18
4회차	2.979	2.982	2.77	0.653	23.41
5회차	2.984	2.98	2.56	0.603	43.88
6회차	2.977	2.981	2.98	0.702	1.67
7회차	2.98	2.983	2.62	0.618	37.51
8회차	2.978	2.983	2.78	0.655	21.97
9회차	2.981	2.978	2.91	0.686	9.31
10회차	2.982	2.98	2.69	0.634	30.93
평균	2.9789	2.9817	2.81	0.662	19.14

	A	B	C	D	E	F	G
1	지자기 단독	상보 필터		RMS		unit	평균적 흔들림
2	58.03677	0.09215		MAG	0.40014	deg	
3	58.46561	0.09817		CF	0.081483	deg	79.6% 감소
4	57.82738	0.06695		Pk-Pk		unit	순간 틈
5	58.29276	0.06444		MAG	2.5675	deg	
6	58.51935	0.07518		CF	0.37136	deg	85.5% 감소
7	57.73788	0.04011		PSD(평균 밀도)		unit	노이즈 에너지 밀도
8	58.46561	0.04915		MAG	0.53	deg²/Hz	
9	57.34302	-0.00724		CF	0.109	deg²/Hz	23.4배 개선
10	57.08429	-0.07533		Root PSD		unit	센서 정밀도
11	58.63091	-0.05006		MAG	0.53	[deg/√Hz]	
12	57.65588	-0.08263		CF	0.109	[deg/√Hz]	4.8배 개선
13	58.62407	-0.05734					
14	58.418	-0.04542					
15	58.46561	-0.03145					
16	58.81053	0.00166					
17	58.81053	0.03285					
18	58.96742	0.07132					
19	58.68271	0.0911					
20	58.81434	0.11735					
21	59.25583	0.16759					
24	58.09151	0.14976					
25	58.24895	0.13992					

노이즈 테스트





타겟 : (0, 0) -> (3, 3) [m]

o 주행 정확도 (오도메트리
및 주행 알고리즘 정확도)

o 직진 / 커브 구간 가속 및
감속

4 결론

Buck Converter & AC Flyback

- DC/DC Converter : 변압기 설계 실패. 시뮬레이션 기반 최대 효율 ACF - **88.35%**, Buck - **86.88%** 달성. 최대 리플 각각 **4.67%**, **4.87%**
- MPPT 알고리즘 : 10W->5W 발전량 감소시 기존 대비 **50%** 이상 전력 발생량 향상
- Converter 실패 제품 제작 실패 : Ferrite Core 기반 변압기 설계 실패(Inductance) 및 요구 사양의 완품 변압기 구매 불가. -> 시판 MPPT 모듈 및 DC/DC 대체. 시뮬레이션으로 유의미한 결과 도출.

구동 및 조향 제어

- 최종 주행 정확도 : 6m 평균 오차 **0.662%**.
- STM32 코어 기준 CPU 활용률 최대 **9.8%**. 평균 **6.13%**.
- 지터 X -> 실시간 제어 확인

5

질의응답

- .
- 안준영: 차량 구동 제어부 및 센싱 설계 담당
- 마한성: 태양전지 제어 MPPT 담당
- 박현재: 전원부 제어기 설계 담당
- 유효조: Pure Pursuit 담당